

東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

AOSP on taped-out RISC-V SoC without RVA22/23

I came back to finish what I started.



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Who am I ?

Ruinland, ctsai390@andestech.com

Employed by Andes Technology.

Currently humbly serving as the chair of System Library Working Group for RISE,







Agenda

- What needs to be done ?
- How exactly ?
- Why do we even care about pre-RVA22 platforms ?
- If we support them, who will need to be involved ?
- When will Android for RISC-V ever be a thing ?

What needs to be done ?

From architecture point of view :

- soong build system
- bionic libc
    - MISC libraries, i.e. libunwind
- ART
- Shipped prebuilts
- (Optional) NDK toolchain for Apps

The “How-s”, for soong

For better abstractions, add CPU variant into RISC-V CPU definition and limit the extensions.

```
--- a/android/arch_list.go
+++ b/android/arch_list.go
@@ -104,6 +104,9 @@ var cpuVariants = map[ArchType][]string{
         "exynos-m1",
         "exynos-m2",
     },
+    Riscv64:    {
+        "andes-ax45",
+        },
     X86:    {},
     X86_64: {},
 }
diff --git a/cc/config/riscv64_device.go b/cc/config/riscv64_device.go
index 47f0de1c8..d0caa2d81 100644
--- a/cc/config/riscv64_device.go
+++ b/cc/config/riscv64_device.go
@@ -55,7 +55,12 @@ var (

     riscv64Cppflags = []string{}

-    riscv64CpuVariantCflags = map[string][]string{}
+    riscv64CpuVariantCflags = map[string][]string{
+        "andes-ax45" : []string {
+        "-march=rv64gc", //TODO: xandes and other things
+        "-mcpu=ax45",
+        },
+    }
 )

 const (
@@ -71,14 +76,21 @@ func init() {
     exportedVars.ExportVariableReferenceDict("Riscv64ArchVariantCflags", riscv64ArchVariantCflagsVar)
     exportedVars.ExportVariableReferenceDict("Riscv64CpuVariantCflags", riscv64CpuVariantCflagsVar)
     exportedVars.ExportVariableReferenceDict("Riscv64CpuVariantLdflags", riscv64CpuVariantLdflags)
+
+    exportedVars.ExportStringListStaticVariable("Riscv64AndesAX45Cflags", riscv64CpuVariantCflags["andes-ax45"])
+    exportedVars.ExportStringListStaticVariable("Riscv64AndesAX45Ldflags", riscv64CpuVariantCflags["andes-ax45"])
 }

 var (
     riscv64ArchVariantCflagsVar = map[string]string{}

-    riscv64CpuVariantCflagsVar = map[string]string{}
+    riscv64CpuVariantCflagsVar = map[string]string{
+        "andes-ax45": "${config.Riscv64AndesAX45Cflags}",
+    }

-    riscv64CpuVariantLdflags = map[string]string{}
+    riscv64CpuVariantLdflags = map[string]string{
+        "andes-ax45": "${config.Riscv64AndesAX45Ldflags}",
+    }
```

The “How-s”, for bionic

Make functions fall back to RVA20 standard, namely, no RVV. At the moment we did our work, it’s easy. Because of slow emulation of RVV on X86, there’s a quick switch :

```
--- a/libc/arch-riscv64/dynamic_function_dispatch.cpp
+++ b/libc/arch-riscv64/dynamic_function_dispatch.cpp
@@ -45,6 +45,7 @@ static __always_inline int ifunc_faccessat(int dir_fd, const char* path, int mod
 }

 static bool have_fast_v() {
+  return false;
```

The “How-s”, for bionic - - libunwind

Libraries are assuming to handle vector register, so we need to take care of them as well.

```
diff --git a/libunwindstack/RegsRiscv64.cpp b/libunwindstack/RegsRiscv64.cpp
index 532b3b1..9fa4ff5 100644
--- a/libunwindstack/RegsRiscv64.cpp
+++ b/libunwindstack/RegsRiscv64.cpp
@@ -35,7 +35,7 @@
 namespace unwindstack {

 uint64_t RegsRiscv64::GetVlenbFromLocal() {
-#if defined(__riscv)
+#if defined(__riscv_vector)
   uint64_t vlenb;
   asm volatile("csrr %0, 0xc22\n" : "=r"(vlenb)::);
   return vlenb;
diff --git a/libunwindstack/include/unwindstack/RegsGetLocal.h b/libunwindstack/include/unwindstack/RegsGetLocal.h
index 86aab97..d9f1993 100644
--- a/libunwindstack/include/unwindstack/RegsGetLocal.h
+++ b/libunwindstack/include/unwindstack/RegsGetLocal.h
@@ -120,8 +120,12 @@ inline __attribute__((__always_inline__)) void AsmGetRegs(void* reg_data) {
       "sd t4, 232(%[base])\n"
       "sd t5, 240(%[base])\n"
       "sd t6, 248(%[base])\n"
+#ifdef __riscv_vector
       "csrr t1, 0xc22\n"
       "sd t1, 256(%[base])\n"
+#else
+      "sd zero, 256(%[base])\n"
+#endif
       "la t1, 1b\n"
       "sd t1, 0(%[base])\n"
       : [base] "+r"(reg_data)
```

The “How-s”, for ART. The BIG headache.

Unlisting things from “basic” features :

```
--- a/runtime/arch/riscv64/instruction_set_features_riscv64.cc
+++ b/runtime/arch/riscv64/instruction_set_features_riscv64.cc
@@ -30,11 +30,7 @@ using android::base::StringPrintf;
 // Basic feature set is rv64gcv_zba_zbb_zbs, aka rv64imafdcv_zba_zbb_zbs.
 constexpr uint32_t BasicFeatures() {
   return Riscv64InstructionSetFeatures::kExtGeneric |
-         Riscv64InstructionSetFeatures::kExtCompressed |
-         Riscv64InstructionSetFeatures::kExtVector |
-         Riscv64InstructionSetFeatures::kExtZba |
-         Riscv64InstructionSetFeatures::kExtZbb |
-         Riscv64InstructionSetFeatures::kExtZbs;
+         Riscv64InstructionSetFeatures::kExtCompressed;
 }
```

The “How-s”, for ART. The BIG headache. (cont.)

Many hardcoded asm under `art/runtime/**riscv64`, define substitutes inside `asm_support_riscv64.S`

The needed ones as we do the work :

```
.macro lea.h rd, rs1, rs2
.macro lea.w rd, rs1, rs2
.macro lea.d rd, rs1, rs2
.macro lea.b.ze rd, rs1, rs2
.macro lea.h.ze rd, rs1, rs2
.macro lea.w.ze rd, rs1, rs2
.macro lea.d.ze rd, rs1, rs2
.macro add.uw rd, rs1, rs2
.macro sh1add rd, rs1, rs2
.macro sh1add.uw rd, rs1, rs2
.macro sh2add rd, rs1, rs2
.macro sh2add.uw rd, rs1, rs2
.macro sh3add rd, rs1, rs2
.macro sh3add.uw rd, rs1, rs2
.macro andn rd, rs1, rs2
.macro bexti rd, rs, shamt
```

substituting with our secrete sauce :-p

```
+#ifndef __riscv_zba
+.macro lea.h rd, rs1, rs2
+    .insn r 0b1011011, 0b000, 0b0000101, \rd, \rs1, \rs2
+.endm
```

```
+.macro sh1add rd, rs1, rs2
+    lea.h \rd, \rs2, \rs1
+.endm
```

The “How-s”, for ART. The BIG headache. (cont.)

The assembler strenuously assumes of Zb* no matter what ..... get it emit our secrete sauce again :-p

```
--- a/compiler/utils/riscv64/assembler_riscv64.cc
+++ b/compiler/utils/riscv64/assembler_riscv64.cc
@@ -1520,38 +1520,31 @@ void Riscv64Assembler::CUnimp() {
 ////////////////////////////// RV64 "Zba" Instructions  START /////////////////////////////

 void Riscv64Assembler::AddUw(XRegister rd, XRegister rs1, XRegister rs2) {
-  AssertExtensionsEnabled(Riscv64Extension::kZba);
-  EmitR(0x4, rs2, rs1, 0x0, rd, 0x3b);
+  EmitR(0b0001000, rs1, rs2, 0b000, rd, 0b1011011);
 }
```

```
--- a/compiler/utils/riscv64/assembler_riscv64.cc
+++ b/compiler/utils/riscv64/assembler_riscv64.cc
@@ -1602,7 +1602,17 @@ void Riscv64Assembler::Cpopw(XRegister rd, XRegister rs1) {
 }

 void Riscv64Assembler::Min(XRegister rd, XRegister rs1, XRegister rs2) {
-  AssertExtensionsEnabled(Riscv64Extension::kZbb);
+  if (!IsExtensionEnabled(Riscv64Extension::kZbb)) {
+    Riscv64Label label, end;
+    Blt(rs1, rs2, &label);
+    Mv(rd, rs2);
+    J(&end);
+    Bind(&label);
+    Mv(rd, rs1);
+    Bind(&end);
+    return;
+  }
+
   EmitR(0x5, rs2, rs1, 0x4, rd, 0x33);
 }
```

The “How-s”, for ART. The BIG headache. (cont.)

Remind the codegen that we don’t have Zb*

```
diff --git a/compiler/optimizing/intrinsics_riscv64.cc b/compiler/optimizing/intrinsics_riscv64.cc
index 3ce6d73fae..845e106bcd 100644
--- a/compiler/optimizing/intrinsics_riscv64.cc
+++ b/compiler/optimizing/intrinsics_riscv64.cc
@@ -380,6 +380,7 @@ static void GenerateReverseBytes(CodeGeneratorRISCV64* codegen,
   }
 }

+#if 0
 static void GenerateReverseBytes(CodeGeneratorRISCV64* codegen,
                                  HInvoke* invoke,
                                  DataType::Type type) {
@@ -472,6 +473,7 @@ void IntrinsicLocationsBuilderRISCV64::VisitShortReverseBytes(HInvoke* invoke) {
 void IntrinsicCodeGeneratorRISCV64::VisitShortReverseBytes(HInvoke* invoke) {
   GenerateReverseBytes(codegen_, invoke, DataType::Type::kInt16);
 }
+#endif

 template <typename EmitOp>
 void EmitIntegralUnOp(HInvoke* invoke, EmitOp&& emit_op) {
@@ -479,6 +481,7 @@ void EmitIntegralUnOp(HInvoke* invoke, EmitOp&& emit_op) {
   emit_op(locations->Out().AsRegister<XRegister>(), locations->InAt(0).AsRegister<XRegister>());
 }

+#if 0
 void IntrinsicLocationsBuilderRISCV64::VisitIntegerBitCount(HInvoke* invoke) {
   CreateIntToIntNoOverlapLocations(allocator_, invoke);
 }
@@ -594,6 +597,7 @@ void IntrinsicCodeGeneratorRISCV64::VisitLongNumberOfTrailingZeros(HInvoke* invo
   Riscv64Assembler* assembler = GetAssembler();
   EmitIntegralUnOp(invoke, [&](XRegister rd, XRegister rs1) { __ Ctz(rd, rs1); });
 }
+#endif

 static void GenerateDivideUnsigned(HInvoke* invoke, CodeGeneratorRISCV64* codegen) {
   LocationSummary* locations = invoke->GetLocations();
@@ -2797,6 +2801,7 @@ void IntrinsicCodeGeneratorRISCV64::VisitJdkUnsafeGetAndSetReference(HInvoke* in
   GenUnsafeGetAndUpdate(invoke, DataType::Type::kReference, codegen_, GetAndUpdateOp::kSet);
 }

+#if 0
 void IntrinsicLocationsBuilderRISCV64::VisitStringCompareTo(HInvoke* invoke) {
   LocationSummary* locations =
       new (allocator_) LocationSummary(invoke,
@@ -3000,6 +3005,7 @@ void IntrinsicCodeGeneratorRISCV64::VisitStringCompareTo(HInvoke* invoke) {
   __ Bind(slow_path->GetExitLabel());
 }
 }
+#endif
```

```
diff --git a/compiler/optimizing/code_generator_riscv64.h b/compiler/optimizing/code_generator_riscv64.h
index b86b30e889..e271e3ff9a 100644
--- a/compiler/optimizing/code_generator_riscv64.h
+++ b/compiler/optimizing/code_generator_riscv64.h
@@ -93,7 +93,23 @@ static constexpr int32_t kFClassNaNMinValue = 0x100;
     V(CRC32UpdateBytes)                                   \
     V(CRC32UpdateByteBuffer)                              \
     V(MethodHandleInvokeExact)                            \
-    V(MethodHandleInvoke)
+    V(MethodHandleInvoke)                                 \
+    V(IntegerReverse)                                     \
+    V(IntegerReverseBytes)                                \
+    V(IntegerBitCount)                                    \
+    V(IntegerHighestOneBit)                               \
+    V(IntegerLowestOneBit)                                \
+    V(IntegerNumberOfLeadingZeros)                        \
+    V(IntegerNumberOfTrailingZeros)                       \
+    V(LongReverse)                                        \
+    V(LongReverseBytes)                                   \
+    V(LongBitCount)                                       \
+    V(LongHighestOneBit)                                  \
+    V(LongLowestOneBit)                                   \
+    V(LongNumberOfLeadingZeros)                           \
+    V(LongNumberOfTrailingZeros)                          \
+    V(ShortReverseBytes)                                  \
+    V(StringCompareTo)
```

```
diff --git a/compiler/optimizing/code_generator_riscv64.cc b/compiler/optimizing/code_generator_riscv64.cc
index 4a96623cda..9336c4dd3a 100644
--- a/compiler/optimizing/code_generator_riscv64.cc
+++ b/compiler/optimizing/code_generator_riscv64.cc
@@ -5349,7 +5349,6 @@ void InstructionCodeGeneratorRISCV64::VisitXor(HXor* instruction) {
 }

 void LocationsBuilderRISCV64::VisitRiscv64ShiftAdd(HRiscv64ShiftAdd* instruction) {
-  DCHECK(codegen_->GetInstructionSetFeatures().HasZba());
   DCHECK_EQ(instruction->GetType(), DataType::Type::kInt64)
       << "Unexpected ShiftAdd type: " << instruction->GetType();
```

Some prebuilts are already built with RVA22 and beyond

For instance : `neuralnetworks_sample_sl_driver_prebuilt.so`

Ask Soong to rebuild it :
`build/soong/soong_ui.bash --make-mode packages/modules/NeuralNetworks/driver/sample_shim/android_riscv64/neuralnetworks_sample_sl_driver_prebuilt.so`

```
ruinland@swapp01:/hdd1/ruinland/aosp/packages/modules/NeuralNetworks/driver/sample_shim/android_riscv64$ readelf -A ./neuralnetworks_sample_sl_driver_prebuilt.so
Attribute Section: riscv
File Attributes
  Tag_RISCV_stack_align: 16-bytes
  Tag_RISCV_arch: "rv64i2p1_m2p0_a2p1_f2p2_d2p2_c2p0_zicsr2p0_zifencei2p0"
```

NDK

The toolchain needs to be reminded about the flags.

```
--- a/build/core/toolchains/riscv64-linux-android-clang/setup.mk
+++ b/build/core/toolchains/riscv64-linux-android-clang/setup.mk
@@ -26,11 +26,13 @@ TARGET_CFLAGS := -fPIC

 TARGET_riscv64_release_CFLAGS := \
     -O2 \
+    -march=rv64gc \
     -DNDEBUG \

 TARGET_riscv64_debug_CFLAGS := \
     -O0 \
     -UNDEBUG \
+    -march=rv64gc \
     -fno-limit-debug-info \
```

```
--- a/clang/lib/Driver/ToolChains/Arch/RISCV.cpp
+++ b/clang/lib/Driver/ToolChains/Arch/RISCV.cpp
@@ -323,8 +323,8 @@ std::string riscv::getRISCVArch(const llvm::opt::ArgList &Args,
     else if (MABI.starts_with_insensitive("ilp32"))
       return "rv32imafdc";
     else if (MABI.starts_with_insensitive("lp64")) {
-      if (Triple.isAndroid())
-        return "rv64imafdcv_zba_zbb_zbs";
+      // if (Triple.isAndroid())
+      //   return "rv64imafdcv_zba_zbb_zbs";

       return "rv64imafdc";
     }
@@ -343,8 +343,8 @@ std::string riscv::getRISCVArch(const llvm::opt::ArgList &Args,
   } else {
     if (Triple.getOS() == llvm::Triple::UnknownOS)
       return "rv64imac";
-    else if (Triple.isAndroid())
-      return "rv64imafdcv_zba_zbb_zbs";
+    // else if (Triple.isAndroid())
+    //   return "rv64imafdcv_zba_zbb_zbs";
     else
       return "rv64imafdc";
   }
```

In app gradle, add into abi build filter :

```
ndk {
abiFilters 'riscv64'
}
```

NDK (cont.)

Google official NDK-samples bugs that we reported - -

- Ruinland : ndk-samples hiccup :

Move Table

ello-oboe	Build FAIL[1]
native-activity	Build FAIL[2][3]
hello-libs	Needs modification on build flow

  - [1] debug|riscv64 : No compatible library found [//oboe/oboe]
  - [2] error: 'ASensorManager_getInstance' is deprecated: first deprecated in Android 26 - Use ASensorManager_getInstanceForPackage instead [-Werror,-Wdeprecated-declarations]
  - [3] error: 'AChoreographer_postFrameCallback' is deprecated: first deprecated in Android 29 - Use AChoreographer_postFrameCallback64 instead [-Werror,-Wdeprecated-declarations]

Why do we even care about pre-RVA22 platforms ?

Economical-wise, users “demand” **cheap** products, that are already tied to AOSP ecosystem.

Smart Kiosk,

Smart POS,

Smart control panel,

.....

all sorts of smart b*llsh*t.





Believe it or not :

Android KitKat is still used in non-upgradable machines.





If we support them, who will need to be involved ?

Google for obvious reason, as mentioned in the slides,

many of the essential components are not cleanly-abstracted.

Architecture-wise, Andes is really interested in maintaining and testing.

SoC-wise, I believe that there are plenty of (testchip) RISC-V SoC running/ran AOSP :

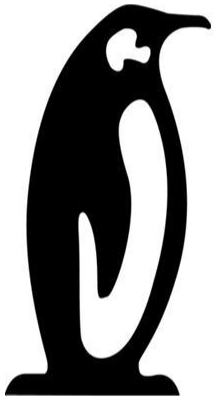
Andes Qilai, Alibaba/Xunatie C910 series, StarFive JH7100

When will Android for RISC-V ever be a thing ?

When the amount of chips are growing =>

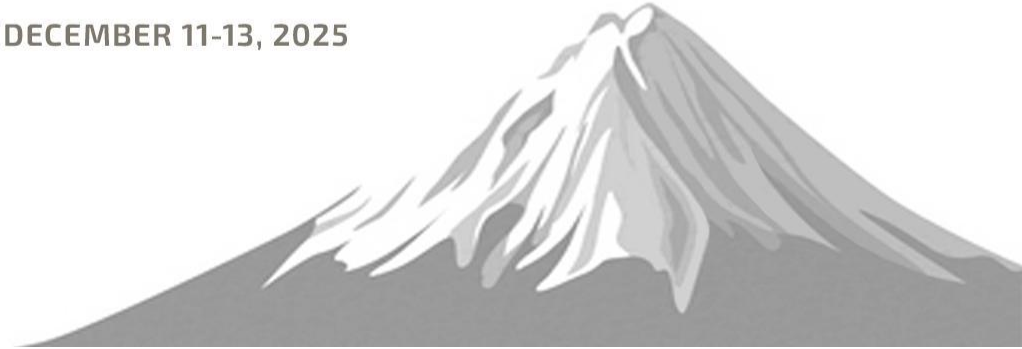
Business-wise : when NDK is more matured and ready. When missing components (e.g. TEE) are there.

Customer-wise : App Store (not necessarily GooglePlay) needs to be ported. 3rd party App store are very popular in mainland China. F-Droid maybe ?



東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025



Backup

Place holders

Place holders